

Diseño Digital Avanzado

Microblaze - GPIO - VIO

Dr. Ariel L. Pola
ariel.pola@mi.unc.edu.ar
December 12, 2025

Tabla de Contenidos

1. Introducción
2. Nuevo Proyecto
3. Creando un nuevo diseño
4. Instancias de periféricos
5. Instancia de VIO
6. Instanciar el uP en Top Level
7. Instanciar el VIO en Top Level
8. Crear la aplicación en Vitis
9. Tunel, Programar la FPGA y Ejecutar la Aplicación en Forma Remota

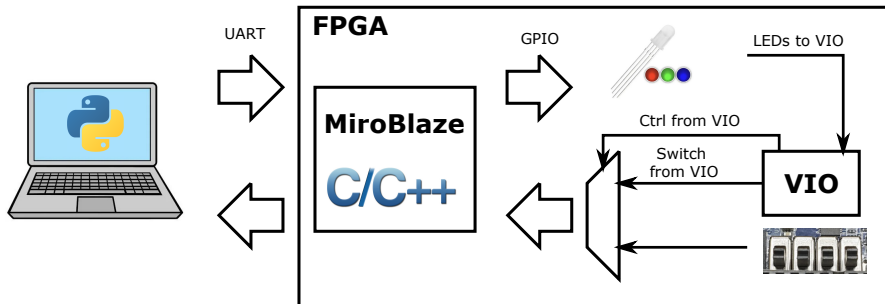
Introducción



Introducción

- El objetivo de esta presentación es detallar paso a paso la instancia y programación de un micro-embebido.
- El proyecto finaliza con el encendido de leds utilizando un script de python ejecutado en la PC y comunicandose con la FPGA utilizando el puerto UART y GPIOs hacia los leds.

Introducción

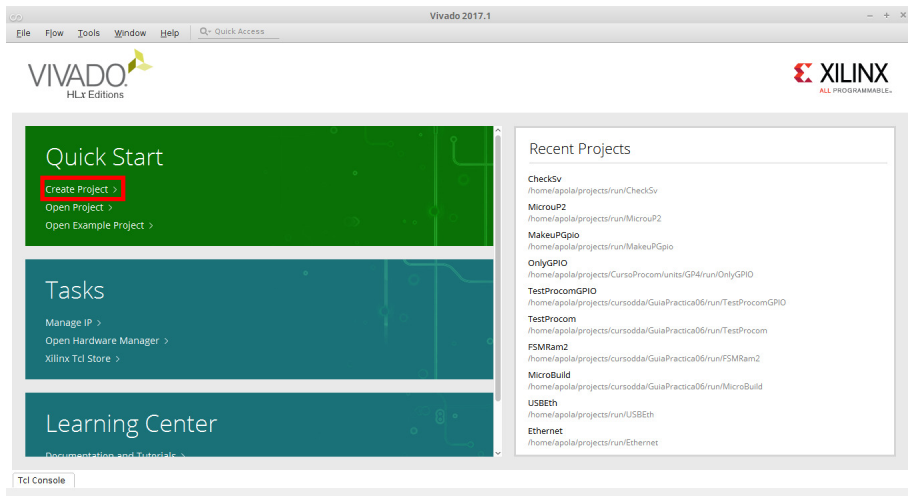


Descripción general.

Nuevo Proyecto



Nuevo Proyecto



Crear un nuevo proyecto.

Nuevo Proyecto

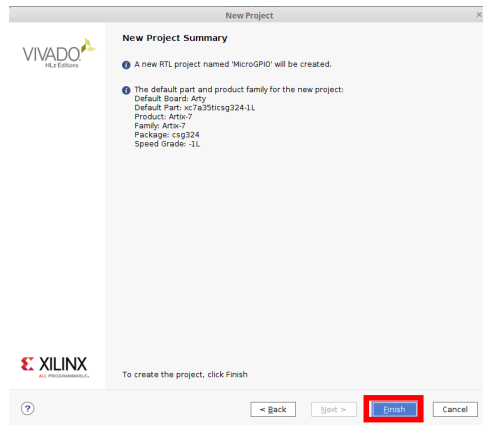
The image displays four sequential screenshots of the Vivado 'New Project' wizard, with red boxes highlighting key input fields and buttons:

- Screenshot 1 (Top Left):** The 'Create a New Vivado Project' screen. It provides instructions on how to create a project and includes a 'Next >' button highlighted with a red box.
- Screenshot 2 (Top Right):** The 'Project Name' screen. It prompts the user to enter a project name and location. The 'Project name' field contains 'MicroGPO' and is highlighted with a red box. The 'Next >' button is also highlighted.
- Screenshot 3 (Bottom Left):** The 'Project Type' screen. It asks the user to specify the type of project to create. The 'RTL Project' option is selected and highlighted with a red box. The 'Next >' button is also highlighted.
- Screenshot 4 (Bottom Right):** The 'Default Part' screen. It prompts the user to choose a default Xilinx part or board. The 'Board' tab is selected and highlighted with a red box. Below, a table of available boards is shown, with the first row highlighted by a red box.

Part	Manufacturer	Part Number	Package	Version
Arty	digilentinc.com	C0	@x7a150xpg234-1	1.1
Cmod A7-150	digilentinc.com	B.0	@x7a150xpg234-1	1.1
Cmod A7-250	digilentinc.com	B.0	@x7a150xpg234-1	1.1
Neopix	digilentinc.com	B.1	@x7a150xpg234-1	1.1
Neopix DDR	digilentinc.com	C.1	@x7a150xpg234-1	1.1

Definir un nombre del proyecto, directorio de trabajo y kit Arty.

Nuevo Proyecto



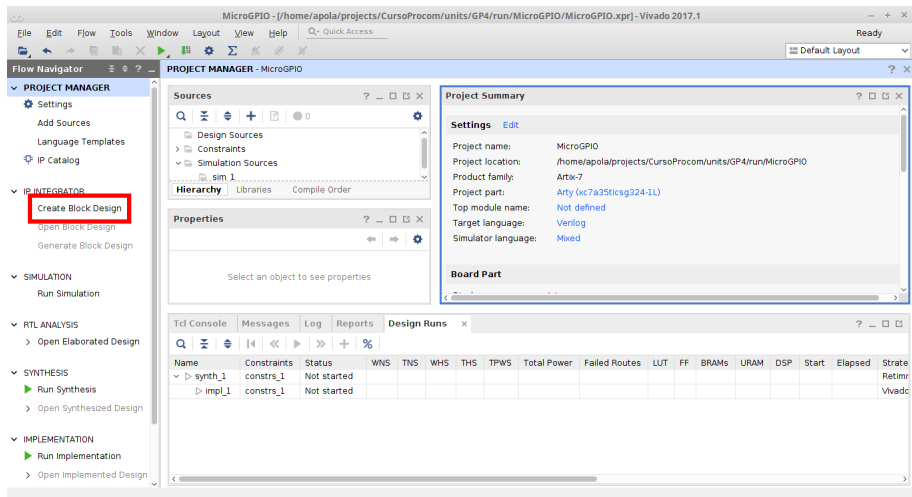
Resumen del proyecto.



The image shows a top-down view of an Intel Agilex FPGA chip. The chip is a square integrated circuit with various components labeled on its surface. At the top, the text 'Intel® Agilex™ FPGA' is printed in blue. Below this, there are several smaller components: a black 'HBM' (High Bandwidth Memory) block, a yellow 'EMIB' (Embedded Multi-Die Interconnect Bridge) block, a central black '10nm FPGA Fabric' block, a grey 'DDR' (Double Data Rate) memory block, a blue 'CPU Cores' block, a green 'PCIe Gen 5' block, a red 'Other Chiplet' block, and a green 'FPGA' block. The chip is set against a dark blue background with a network of glowing blue lines and yellow nodes, suggesting a digital or network environment.

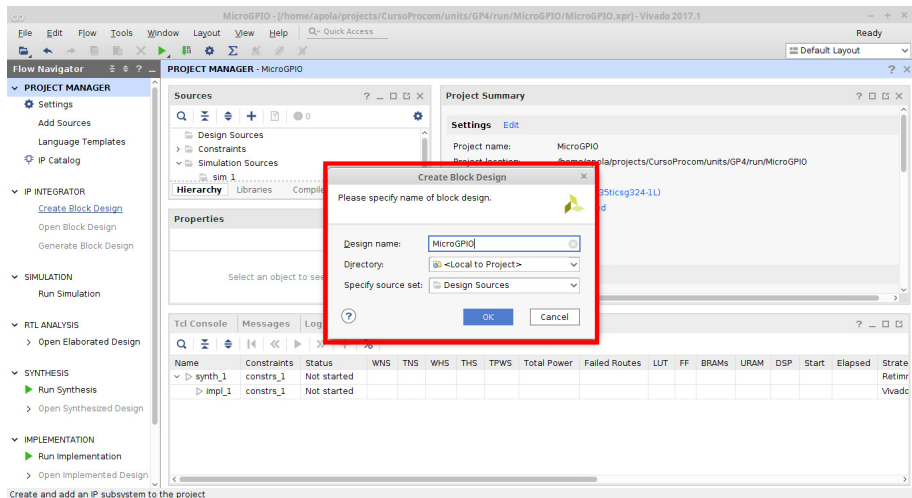
Creando un nuevo diseño

Creando un nuevo diseño



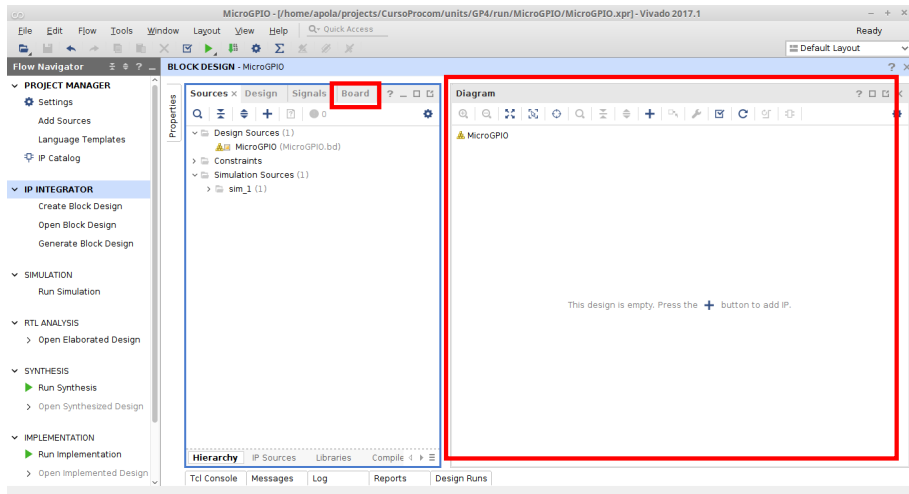
Crean un nuevo bloque.

Creando un nuevo diseño



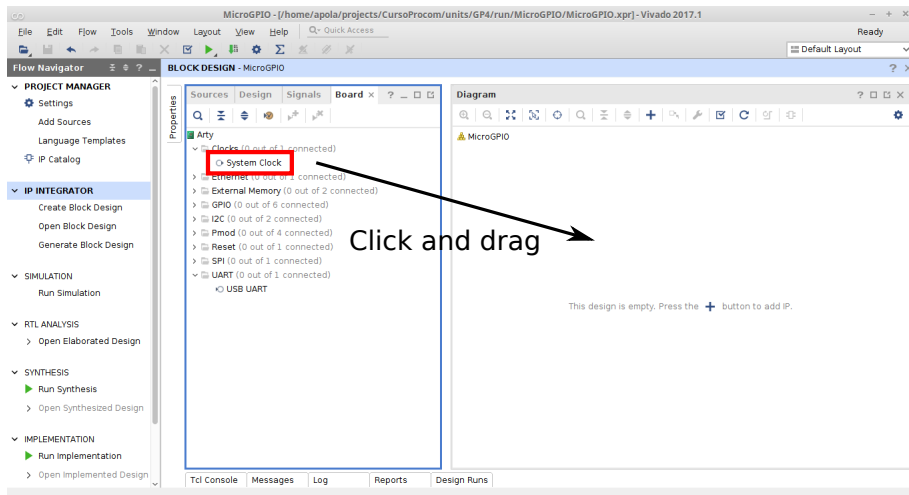
Definir el nombre del nuevo diseño y el directorio de trabajo.

Creando un nuevo diseño



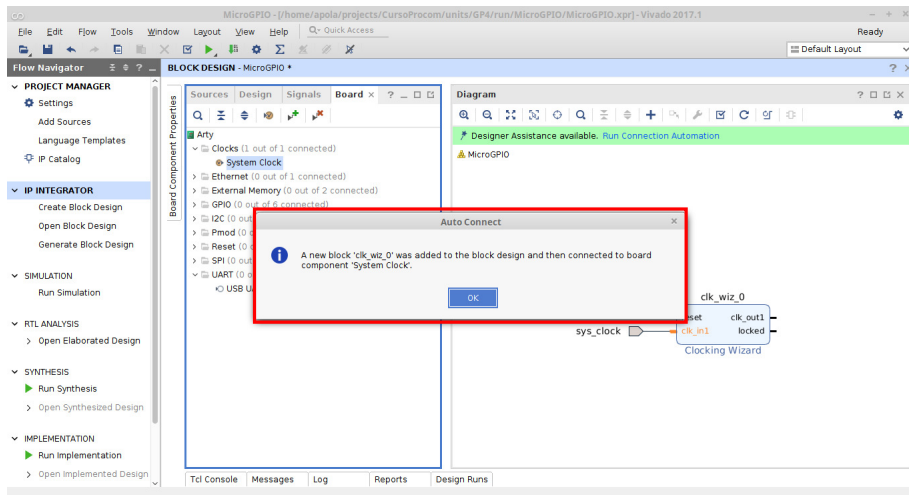
Nueva ventana de trabajo denominada “Diagrama” y en la pestaña “Board” se definen todos los componentes de la FPGA seleccionada.

Creando un nuevo diseño



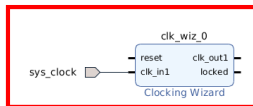
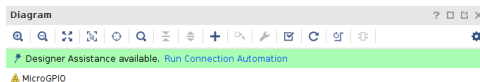
Instanciar “System Clock” haciendo click y arrastrando hacia la ventana.

Creando un nuevo diseño



Mensaje de instancia de bloque.

Creando un nuevo diseño



Double Click

Hacer doble click sobre "Clocking Wizard".

Creando un nuevo diseño

Re-customize IP

Clocking Wizard (5.4)

Documentation IP Location

IP Symbol Resource

☐ Show disabled ports

Component Name clk_wiz_0

Board Clocking Options **Output Clocks** MMCM Settings Summary

The phase is calculated relative to the active input clock.

Output Clock	Port Name	Output Freq (MHz)	Requested	Actual	Phase (degrees)	Requested	Actual	Duty Cycle (%)	Requested
☒ clk_out1	clk_out1	100.000	100.000	0.000	0.000	50.000			
☐ clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000			
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000			
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000			
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000			
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000			
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000			

☐ USE CLOCK SEQUENCING

Clocking Feedback

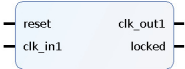
Output Clock	Sequence Number
clk_out1	1
clk_out2	1

Source Signaling

☒ Automatic Control On-Chip ☒ Single-ended

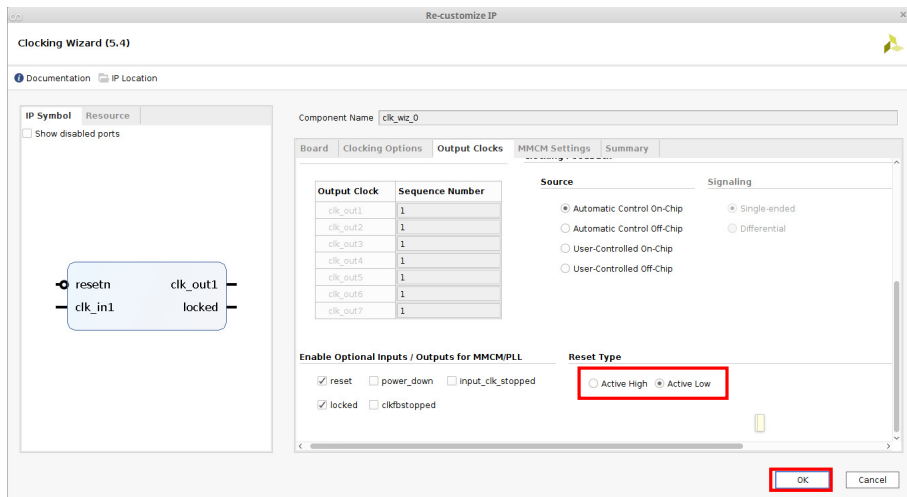
☐ Automatic Control Off-Chip ☐ Differential

OK Cancel



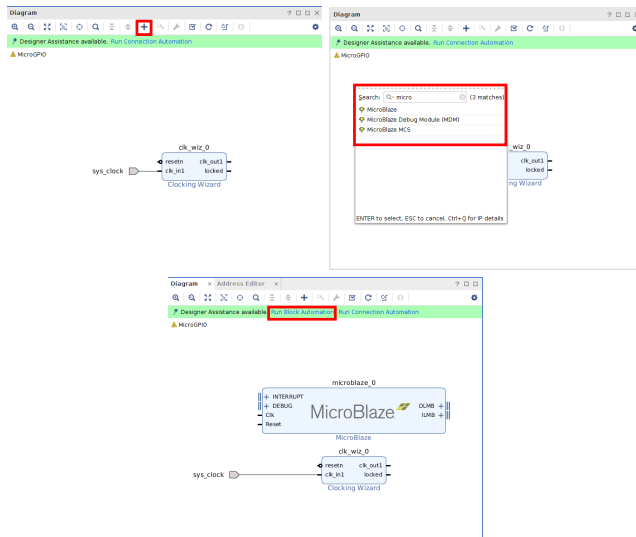
En la pestaña "Output Clocks" asignar la frecuencia de salida en 100MHz.

Creando un nuevo diseño



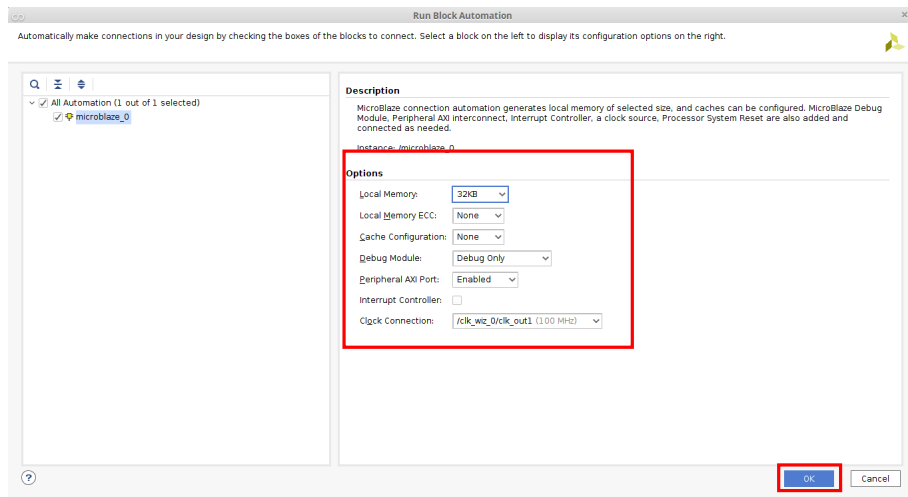
El reset del bloque tiene que ser activo por bajo.

Creando un nuevo diseño



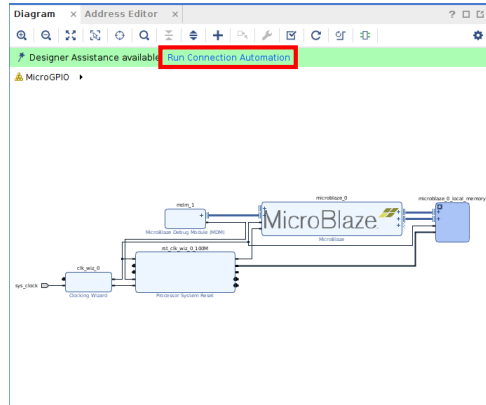
Incluir el core del MicroBlaze y ejecutar "Run Block Automation".

Creando un nuevo diseño



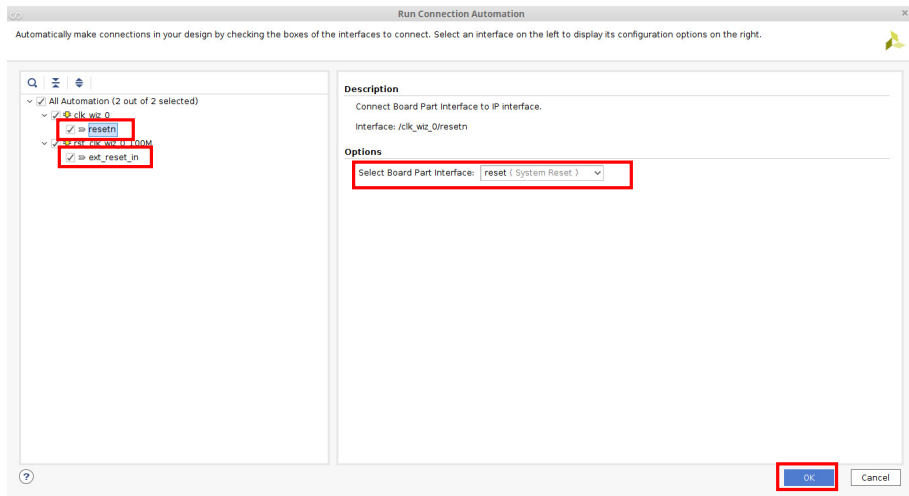
Signar los parámetros de diseño del micro.

Creando un nuevo diseño



Auto-conectar los elementos utilizando “Run Connection Automation”.

Creando un nuevo diseño



Verificar que los dos puertos de reset estén conectados al reset del sistema.

Instancias de periféricos



Instancias de periféricos

MicroGPIO - [/home/apola/projects/CursoProcom/units/GP4/run/MicroGPIO/MicroGPIO.xpr] - Vivado 2017.1

File Edit Flow Tools Window Layout View Help

Ready

Default Layout

Flow Navigator

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Open Elaborated Design

SYNTHESIS

- Run Synthesis
- Open Synthesized Design

IMPLEMENTATION

- Run Implementation
- Open Implemented Design

Block Pin Properties

Sources Design Signals Board

Arty

- Clocks (1 out of 1 connected)
- System Clock
- Ethernet (0 out of 1 connected)
- External Memory (0 out of 2 connected)
- GPIO (0 out of 6 connected)
- I2C (0 out of 2 connected)
- Pmod (0 out of 4 connected)
- Reset (1 out of 1 connected)
- SPI (0 out of 1 connected)
- UART (0 out of 1 connected)
- USB UART

Diagram

MicroGPIO

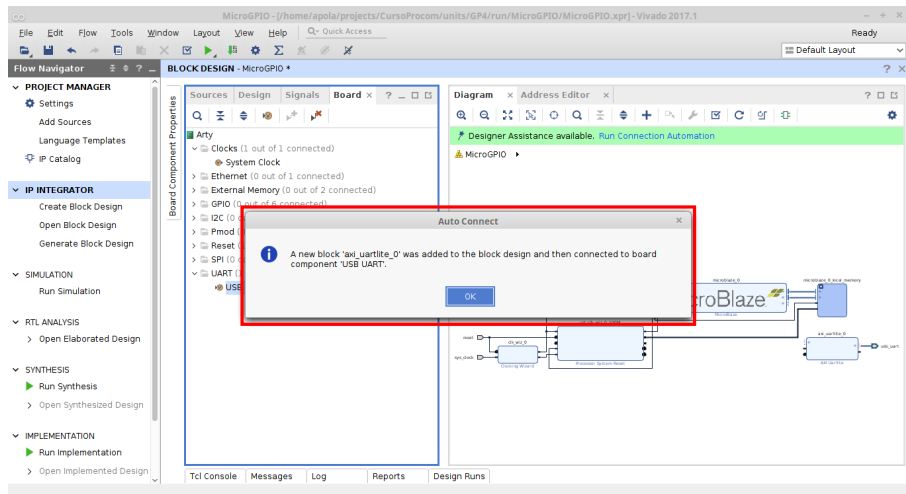
Click and Drag

Tcl Console Messages Log Reports Design Runs

Block Pin: locked

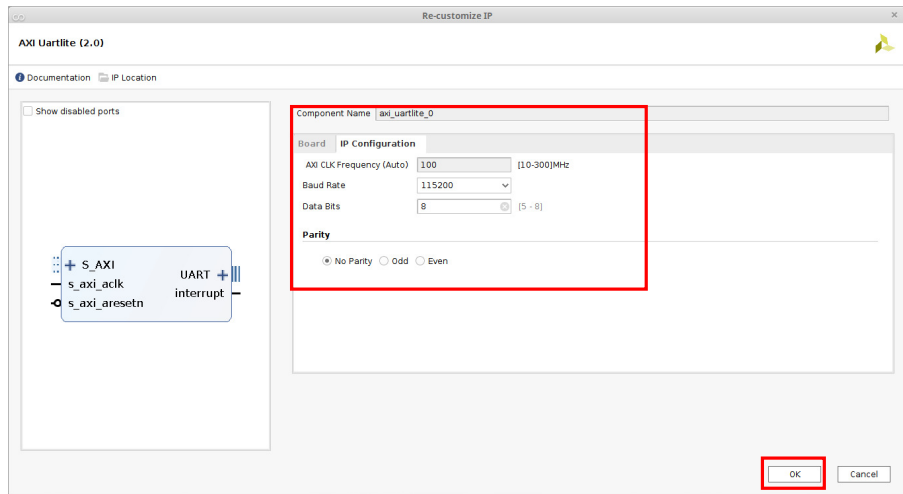
Instanciar el IP USB UART.

Instancias de periféricos



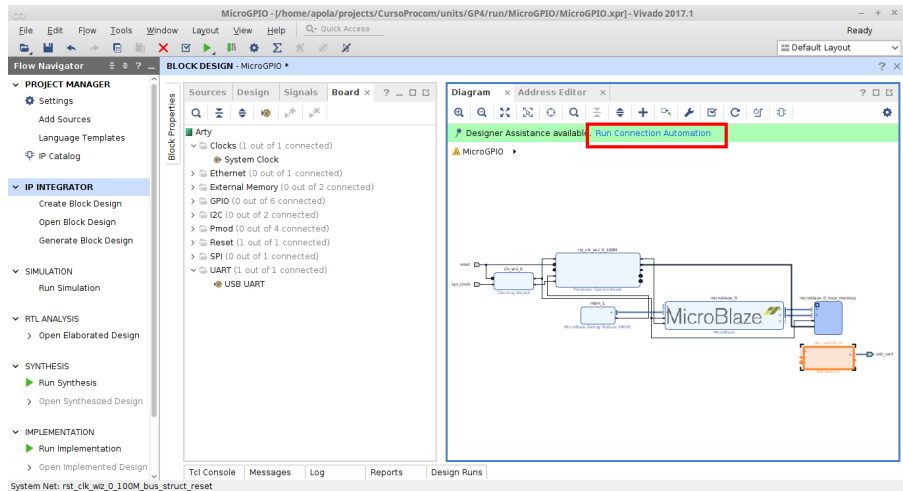
Verificar la instancia.

Instancias de periféricos



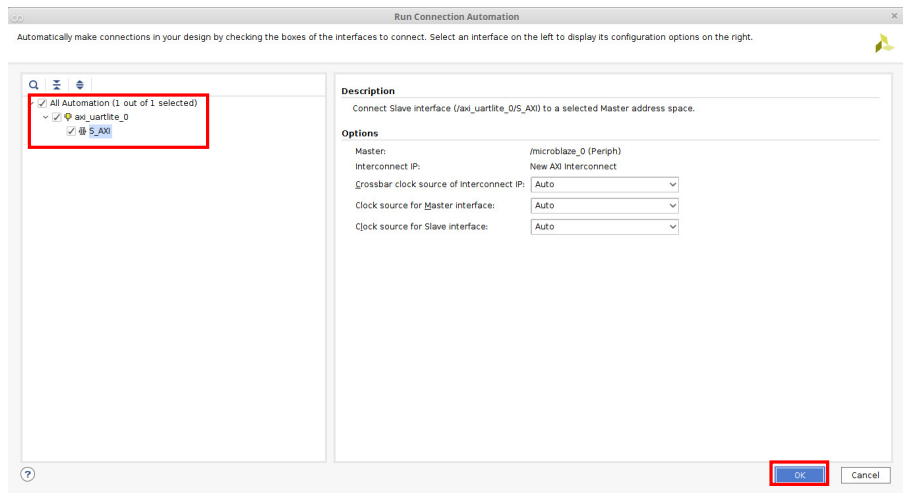
Hacer doble click sobre el periférico UART y configurar el Baud Rate en 115200.

Instancias de periféricos



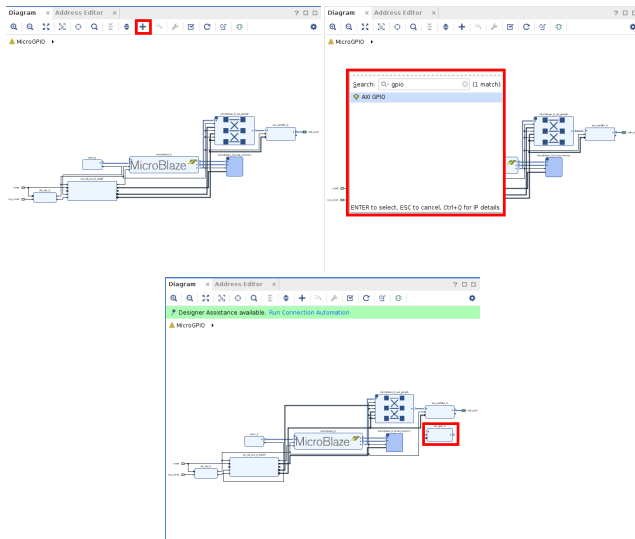
Seleccionar la conexión automática.

Instancias de periféricos



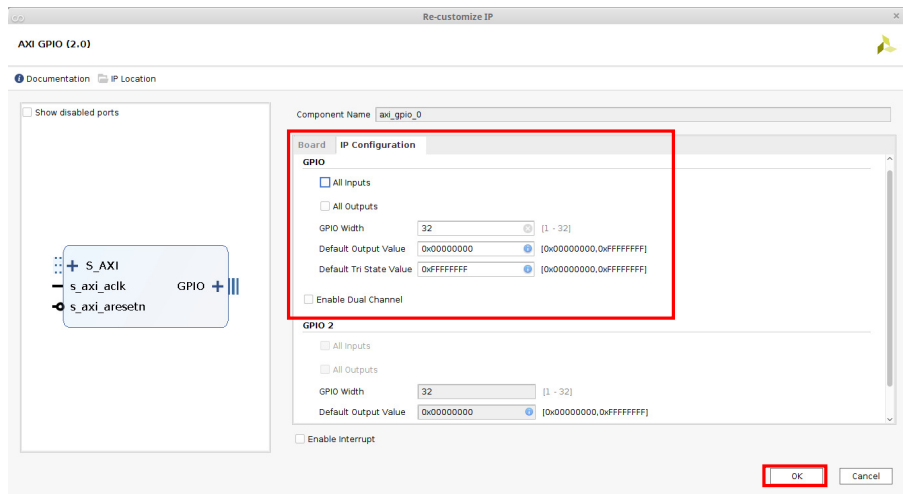
Verificar que todos los puertos estén seleccionados.

Instancias de periféricos



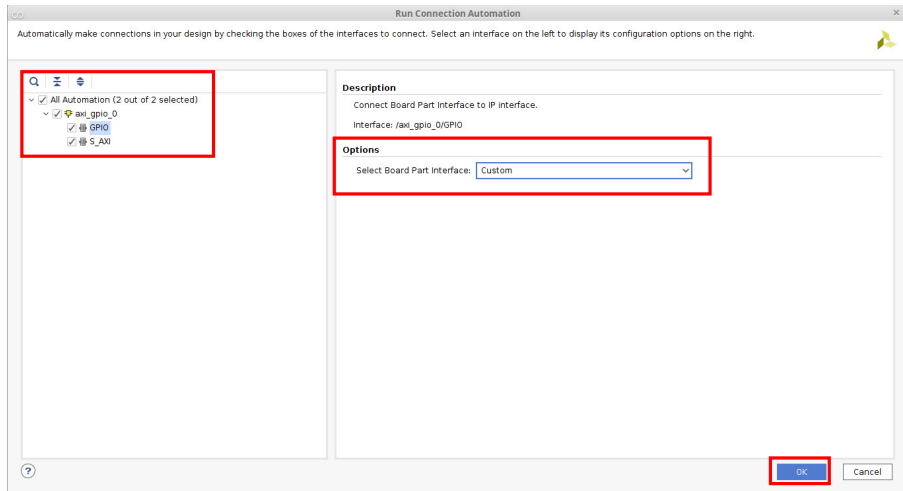
Agregar un nuevo componente GPIO y hacer doble click sobre el IP.

Instancias de periféricos



Verificar la configuración del puerto.

Instancias de periféricos



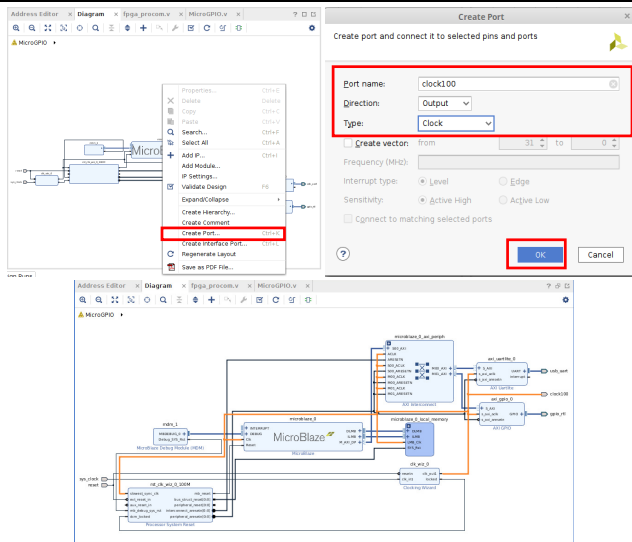
Seleccionar “Run Connection Automation” y verificar que el puerto de salida sea “Custom”.

The diagram illustrates a MicroBlaze system architecture. At the center is the **MicroBlaze** core. To its left is the **MicroBlaze Debug Module (MDM)**, which includes a **Debug Sys_Rst** block. Below the MDM is the **Processor System Reset** block, which contains a **rst_clk_wiz_0_100M** block and a **clk_wiz_0** block. The **clk_wiz_0** block is connected to **reset** and **sys_dock** inputs. The **Processor System Reset** block provides **slowest_sysnc_clk**, **ext_reset_in**, **aux_reset_in**, **mb_debug_sys_rst**, and **dcm_locked** signals to the MicroBlaze core. The MicroBlaze core has **INTERRUPT + DEBUG**, **Clk**, and **Reset** inputs. It is connected to **microblaze_0_local_memory**, which includes **DLSB**, **SRB**, **LMB_Clk**, and **SYS_Rst** blocks. The MicroBlaze core also has **DLSB**, **SRB**, **LMB_Clk**, and **SYS_Rst** outputs. The MicroBlaze core is connected to the **microblaze_0_axi_periph** block, which includes **S00_AXI**, **S00_ACLK**, **S00_ARCLK**, **M00_AXI**, **M00_ARCLK**, **M01_AXI**, and **M01_ARESETN** signals. The **microblaze_0_axi_periph** block is connected to the **AXI Interconnect** block. The **AXI Interconnect** block is connected to the **axi_uartlite_0** block, which includes **S_AXI**, **S_AXI_ACLK**, **S_AXI_ARESETN**, and **UART** signals. The **axi_uartlite_0** block is connected to the **usb_uart** block. The **AXI Interconnect** block is also connected to the **axi_gpio_0** block, which includes **S_AXI**, **S_AXI_ACLK**, **S_AXI_ARESETN**, and **GPIO** signals. The **axi_gpio_0** block is connected to the **gpio_rtl** block.



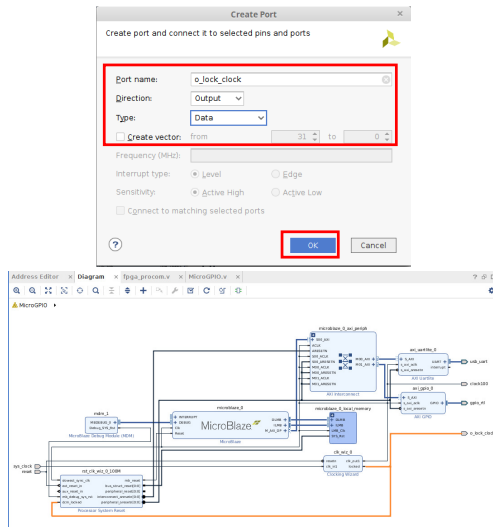
Fundación
FULGOR

Instancias de periféricos



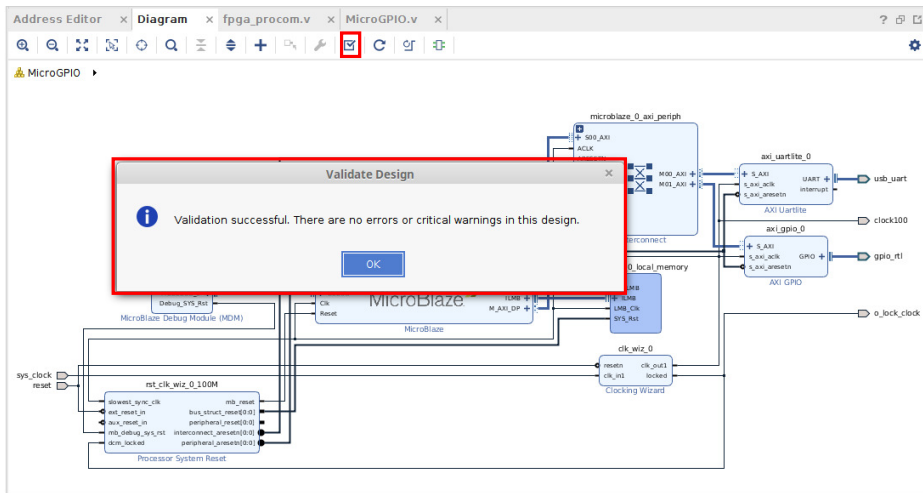
Incluir un nuevo puerto de clock de salida.

Instancias de periféricos



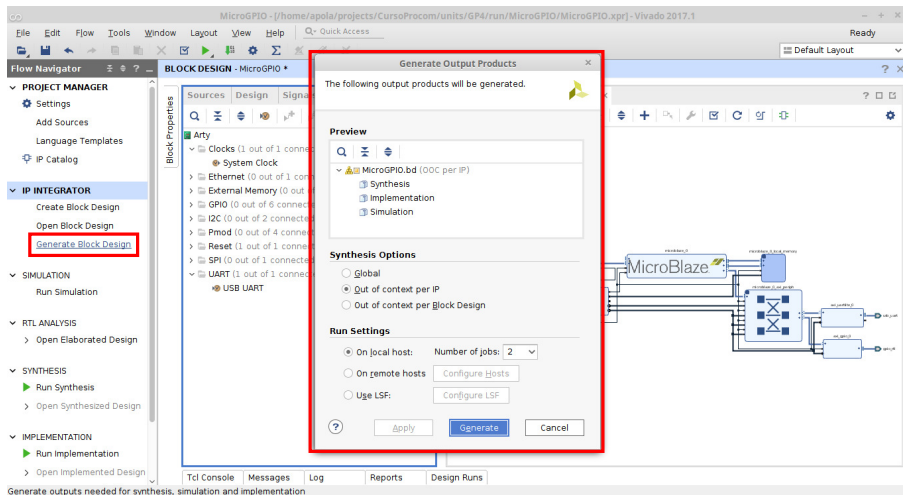
Repetir el paso anterior incluyendo un nuevo puerto de lock de clock.

Instancias de periféricos



Validar el diseño.

Instancias de periféricos



Generar el diseño completo.



Instancia de VIO



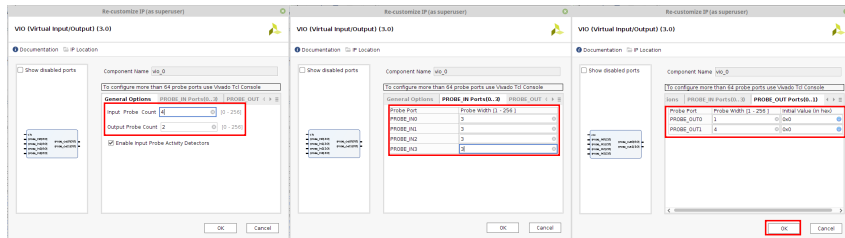
Instancia de VIO

The image shows a sequence of steps in the Xilinx IDE:

- Flow Navigator:** The 'IP INTEGRATOR' section is expanded, and 'Create Block Design' is highlighted with a red box.
- Create Block Design (as superuser):** A dialog box where 'Design name' is set to 'vio' (highlighted with a red box) and 'OK' is pressed (button highlighted with a red box).
- Diagram:** The main workspace is empty, with a message 'This design is empty. Press the + button to add IP.' The '+' button is highlighted with a red box.
- Search:** A search bar shows 'Q: vio' with '(1 match)' and 'VIO (Virtual Input/Output)' listed below it, highlighted with a red box.
- Diagram:** The 'VIO (Virtual Input/Output)' block is placed in the workspace, labeled 'vio_0'. Its ports are 'clk', 'probe_in0[0:0]', and 'probe_out0[0:0]'. The block is highlighted with a red box.

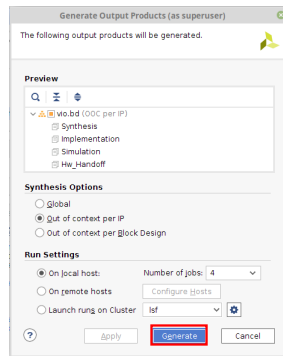
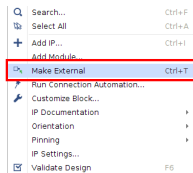
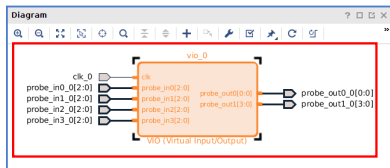
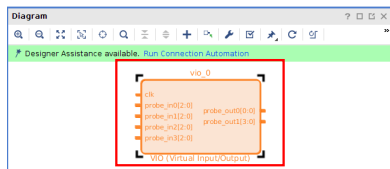
En caso de necesitar controlar en forma remota la FPGA debemos incluir el bloque VIO a nuestro diseño. Creamos un nuevo bloque, le asignamos el nombre, agregamos un nuevo IP, seleccionamos el IP VIO y hacemos doble click sobre el IP generado.

Instancia de VIO



En el ejemplo vamos a habilitar el control externo, controlar los SWITCH y observar el valor de los leds RGB. Agregamos 4 puertos de entrada y 2 de salida. Cada puerto de entrada tiene 3 bits. En el caso de los puertos de salida se tienen 1 y 4 bits respectivamente.

Instancia de VIO

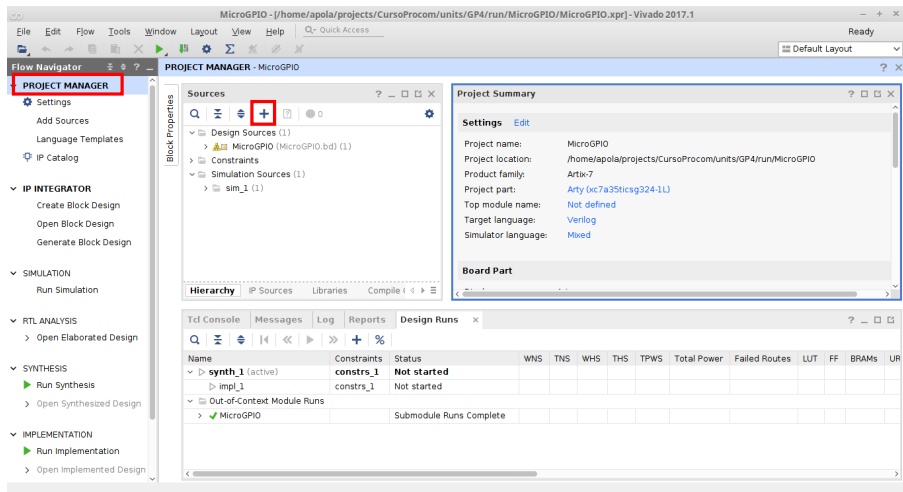


Sobre el bloque modificado hacemos click con el botón derecho, seleccionamos Make External lo cual agrega los puertos exteriores y por último generamos el bloque.

Instanciar el uP en Top Level

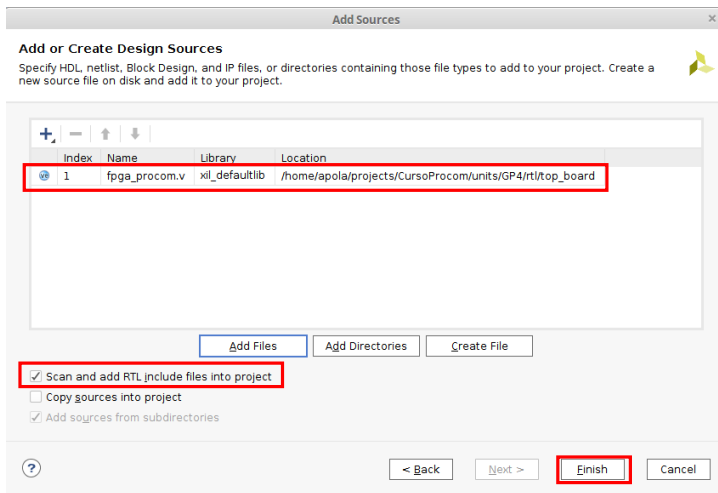


Instanciar el uP en Top Level



Agregar un nuevo archivo verilog.

Instanciar el uP en Top Level



Seleccionar el archivo "fpga_dda" y asegurarse de activar el escaneo de includes.

Instanciar el uP en Top Level

The screenshot displays the Vivado 2017.1 IDE interface for a project named 'MicroGPIO'. The 'Sources' pane on the left shows the project hierarchy, with 'fpga' and 'u_micro' files highlighted. The 'Project Summary' pane on the right shows the Verilog code for 'MicroGPIO'. The 'Design Runs' table at the bottom shows the synthesis status.

Sources:

- Design Sources (3)
 - Verilog Header (2)
 - fpga (fpga_procom.v) (1)
 - u_micro : MicroGPIO (MicroGPIO.bd) (1)
 - MicroGPIO (MicroGPIO.v) (9)

Project Summary:

```
//design 1
MicroGPIO
u_micro
(.clock100 (clockdsp ), // Clock aplicacion
 .gpio_rtl_tri_o (gpo0 ), // GPIO
 .gpio_rtl_tri_i (gp10 ), // GPIO
 .reset (in_reset ), // Hard Reset
 .sys_clock (clk100 ), // Clock de FPGA
 .o_lock_clock (locked ), // Serial Lock Clock
 .usb_uart_rxd (in_rx_uart ), // UART
 .usb_uart_txd (out_tx_uart ) // UART
);

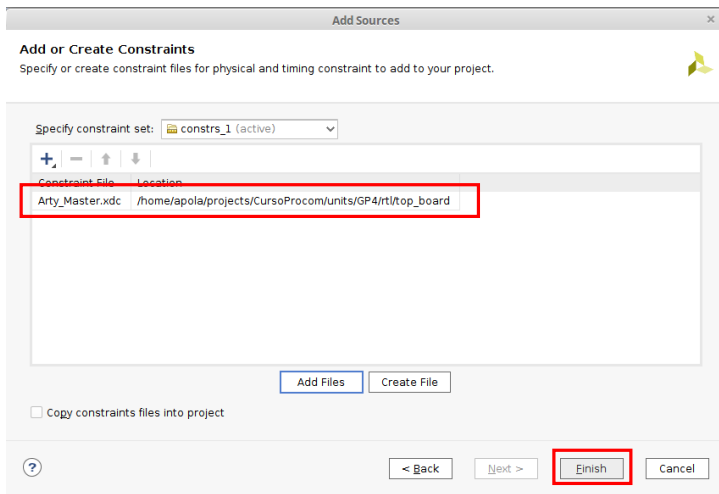
////////////////////////////////////
// Leds
////////////////////////////////////
section not locked = locked;
```

Design Runs:

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMS	UR
synth_1 (active)	constrs_1	Not started											
impl_1	constrs_1	Not started											
Out-of-Context Module Runs													
MicroGPIO		Submodule Runs Complete											

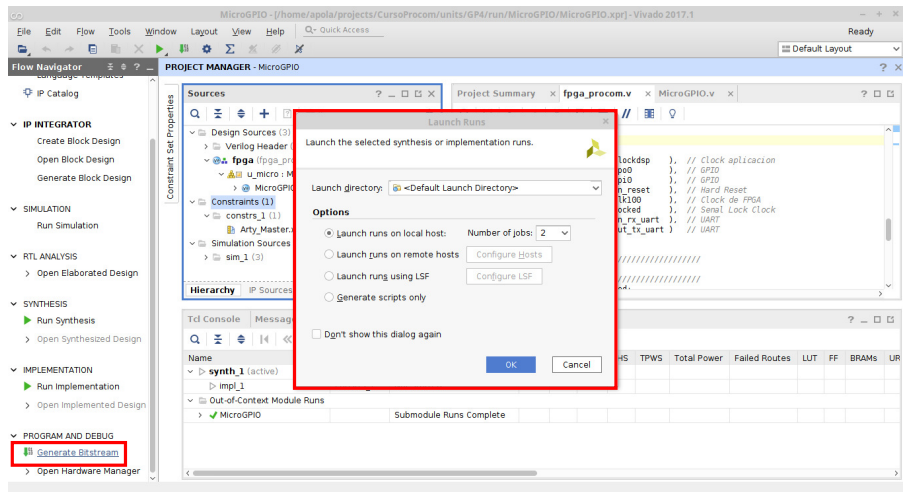
Verificar que el nombre del archivo del micro sea el mismo que se utiliza en el top level y los mismos puertos.

Instanciar el uP en Top Level



Agregar el archivo de puertos físicos.

Instanciar el uP en Top Level



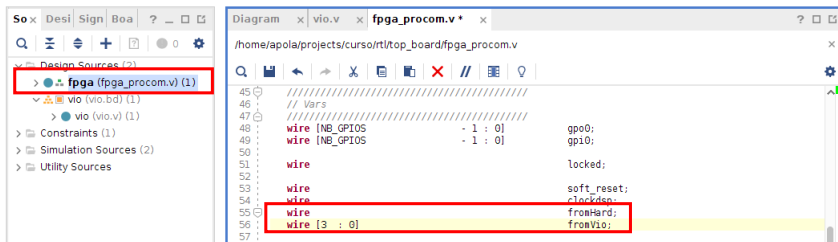
Generar el binario.

Instanciar el VIO en Top Level



Instanciar el VIO en Top Level

- En caso que se esté trabajando en forma remota, vamos a necesitar incluir el VIO para controlar los switch y observar los valores de los puertos de salida.



Declarar en el módulo Top dos variables wire de 1 y 4 bits respectivamente. La primera nos permitirá seleccionar de donde queremos controlar el dispositivo y la segunda es para emular el comportamiento de los switch en forma virtual.

Instanciar el VIO en Top Level

The screenshot displays the Xilinx IDE interface with two windows open. The top window, titled 'Diagram x vio.v', shows the source code for the 'vio' module. The bottom window, titled 'Diagram x fpga_procom.v', shows the source code for the 'fpga_procom' module, which is the top-level design.

Top Window: vio.v

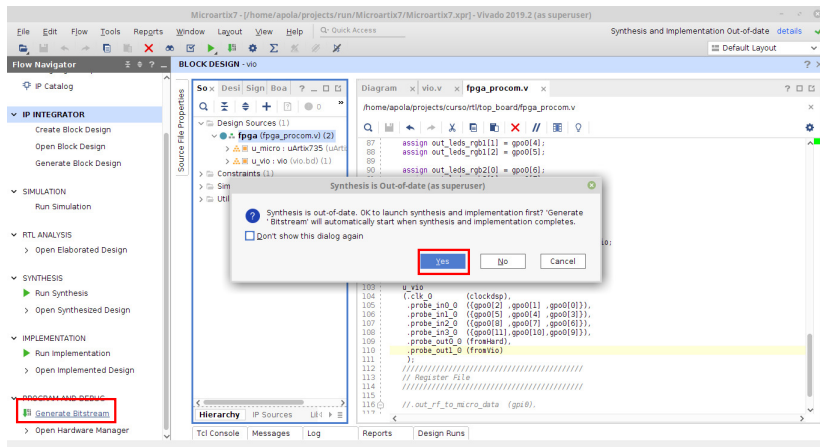
```
timescale 1 ps / 1 ps
// Xilinx Integrator, (x_ipVendor-xilinx.com, x_ipLibra
module vio
    (clk_0,
     probe_in0_0,
     probe_in1_0,
     probe_in2_0,
     probe_in3_0,
     probe_out0_0,
     probe_out1_0);
    xilinx.com:signal:clock:1.0 CLK_CLK_0 CLK" *) (* X_Ib
    input [2:0]probe_in0_0;
    input [2:0]probe_in1_0;
    input [2:0]probe_in2_0;
    input [2:0]probe_in3_0;
```

Bottom Window: fpga_procom.v

```
assign out_leds_rgb1[1] = gpo0[4];
assign out_leds_rgb1[2] = gpo0[5];
assign out_leds_rgb2[0] = gpo0[6];
assign out_leds_rgb2[1] = gpo0[7];
assign out_leds_rgb2[2] = gpo0[8];
assign out_leds_rgb3[0] = gpo0[9];
assign out_leds_rgb3[1] = gpo0[10];
assign out_leds_rgb3[2] = gpo0[11];
assign gpo0[3 : 0] = (fromHard) ? i_sw : fromVio;
//assign gpo0[3 : 0] = i_sw;
assign gpo0[31 : 4] = {28{1'b0}};
vio
u_vio
    (.clk_0 (clockdsp),
     .probe_in0_0 ((gpo0[12], gpo0[11], gpo0[10])),
     .probe_in1_0 ((gpo0[5], gpo0[4], gpo0[3])),
     .probe_in2_0 ((gpo0[8], gpo0[7], gpo0[6])),
     .probe_in3_0 ((gpo0[11], gpo0[10], gpo0[9])),
     .probe_out0_0 (fromHard),
     .probe_out1_0 (fromVio)
    );
```

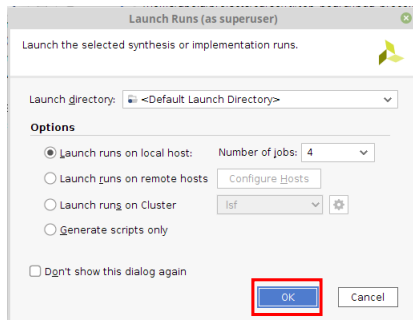
Además, debemos copiar la declaración del módulo VIO e incluirlo en el Top Level. Por último, asignamos la selección del control por medio del bloque VIO y comentamos la asignación directa de los switch.

Instanciar el VIO en Top Level



Generamos el binario nuevamente.

Instanciar el VIO en Top Level

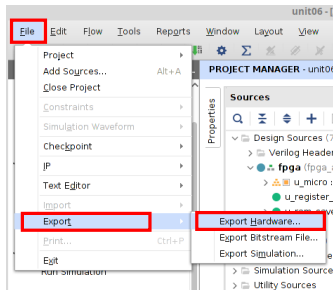


Configuramos cuantos cores queremos utilizar para generar el binario.

Crear la aplicación en Vitis

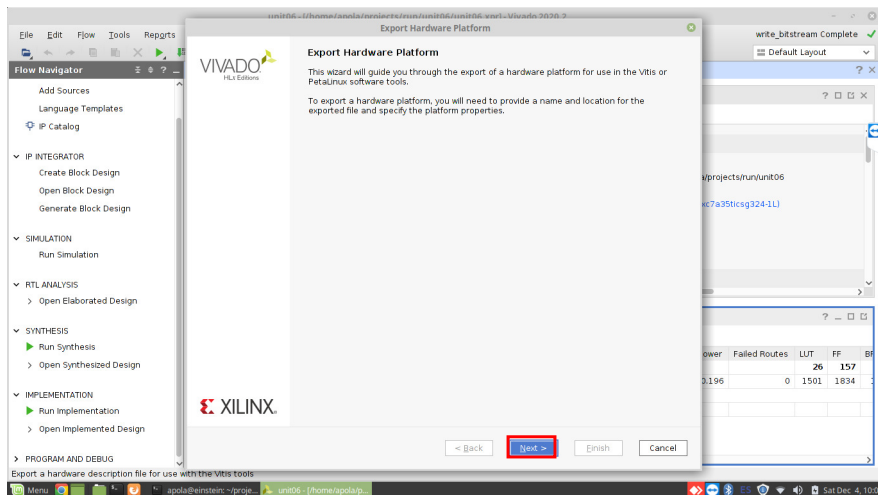


Crear la aplicación en Vitis



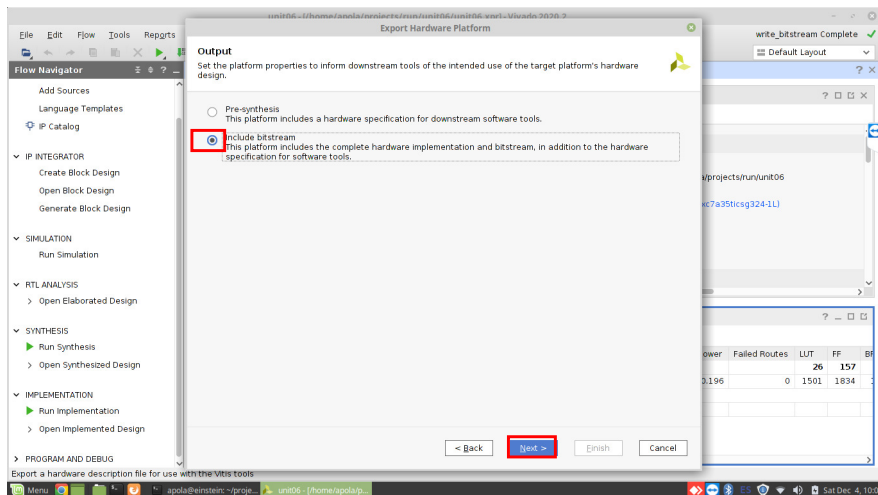
Exportar el hardware desde la opción “File” de la barra de herramientas.

Crear la aplicación en Vitis



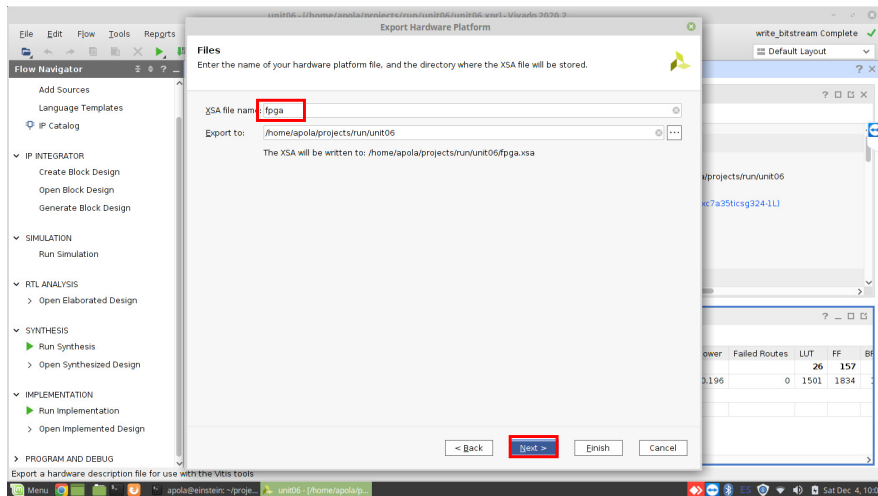
Pulsar NEXT.

Crear la aplicación en Vitis



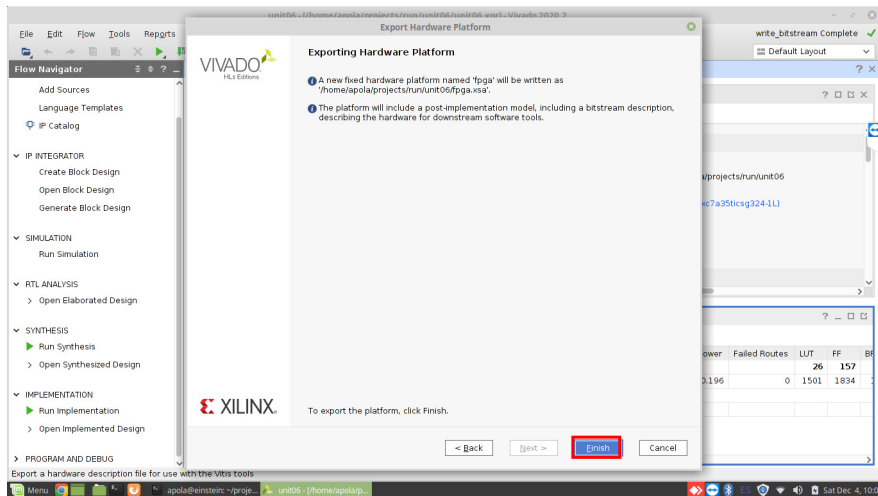
Incluir el bitstream.

Crear la aplicación en Vitis



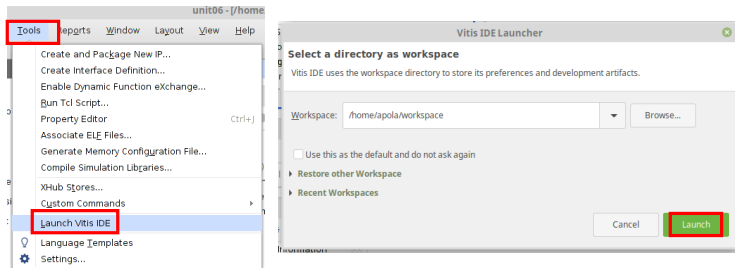
Definir el nombre del XSA (dejarlo por defecto).

Crear la aplicación en Vitis



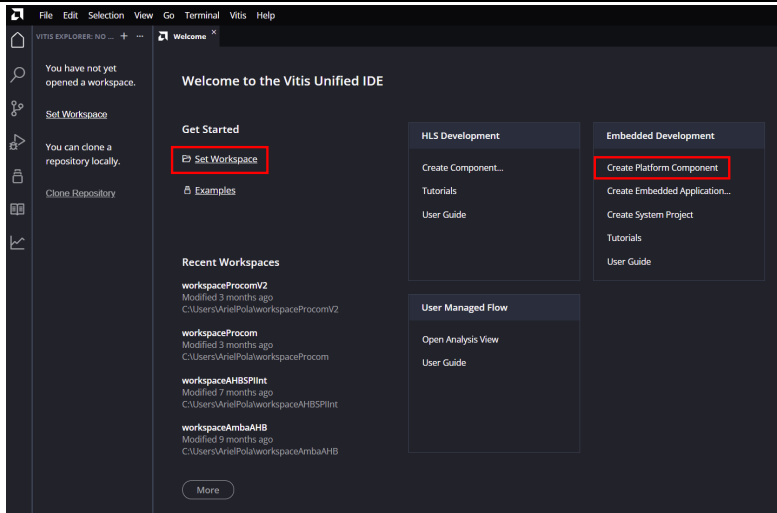
Pulsar FINISH.

Crear la aplicación en Vitis



Ejecutar Vitis desde Vivado, seleccionando la opción Launch Vitis IDE.

Crear la aplicación en Vitis



Entorno de trabajo en Vitis. Configurar el workspace, en donde seleccionamos una carpeta de referencia y luego creamos una plataforma.

Crear la aplicación en Vitis

The image displays four sequential screenshots of the 'Create Platform Component' wizard in Vitis, illustrating the steps to create a new platform component.

Screenshot 1: Name and Location
This screen shows the 'Name and Location' step. The 'Component Name' field is set to 'hello' and the 'Component Location' is set to 'C:\Users\Andrius\workspace'. The 'Next' button is highlighted.

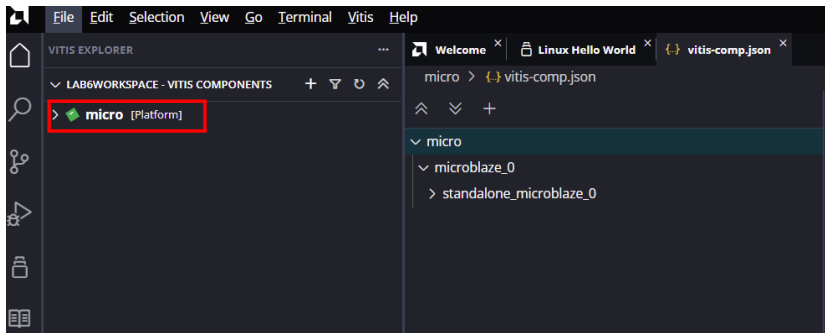
Screenshot 2: Select Platform Creation Flow
This screen shows the 'Select Platform Creation Flow' step. The 'Hardware Design' radio button is selected. The 'Hardware Design (XSA) For Implementation' dropdown is set to 'Select hardware design'. The 'Next' button is highlighted.

Screenshot 3: Select Operating System and Processor
This screen shows the 'Select Operating System and Processor' step. The 'Operating System' dropdown is set to 'standalone' and the 'Processor' dropdown is set to 'microblaze_0'. The 'Next' button is highlighted.

Screenshot 4: Summary
This screen shows the 'Summary' step, displaying the configuration details for the new platform component:
Name: hello
Workspace: C:\Users\Andrius\workspace
Component directory: C:\Users\Andrius\workspace\hello
Operating system: standalone
Processor: microblaze_0
A note at the bottom states: 'Note: A domain with selected operating system and processor will be added to the platform. The platform can be modified later to add new domains or change settings.'

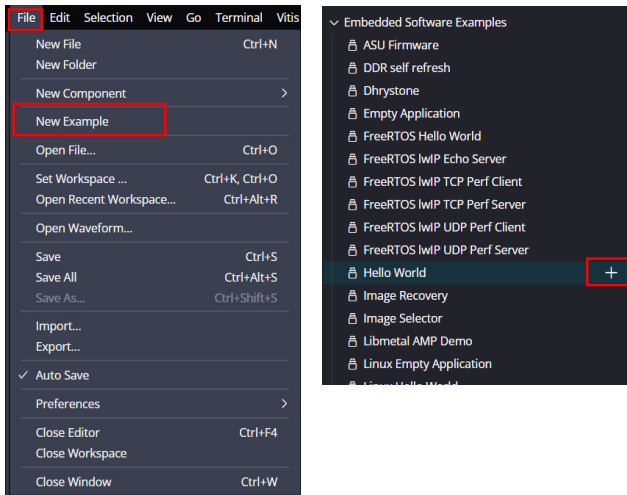
1- Definir un nombre del componente y seleccionar el lugar de destino del proyecto (preferentemente no cambiar de workspace). 2- Seleccionar el XSA generado en Vivado. 3- Revisar las configuraciones.

Crear la aplicación en Vitis



Componente generado.

Crear la aplicación en Vitis



Agregar un nuevo ejemplo de Software (Hello Word), donde se debe hacer click sobre el símbolo '+' para la configuración del software.

Crear la aplicación en Vitis

The image displays four sequential screenshots of the Vitis GUI, illustrating the process of creating a new application component.

Screenshot 1: Create Application Component - Linux Hello World
The 'Name and Location' tab is active. The 'Component name' field is set to 'tspw' (highlighted with a red box). The 'Component location' is set to 'C:\Users\W\OneDrive\Workspace'. A 'Next' button is highlighted with a red box at the bottom right.

Screenshot 2: Create Application Component - Linux Hello World
The 'Select Platform' tab is active. A table lists available platforms. The 'tspw' platform is selected and highlighted with a red box.

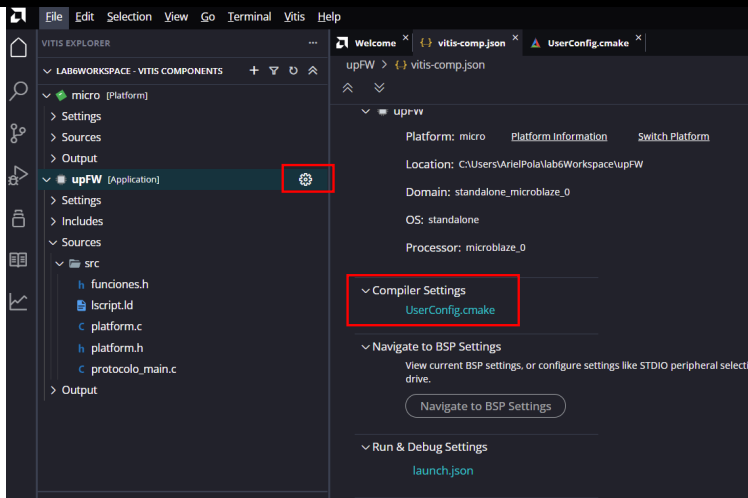
NAME	BOARD	FLOW	VENDOR	PATH
✓ C:\Users\W\OneDrive\Workspace\components\tspw				...
(0) tspw	arty a7 35	ContextIO	virtu.com	C:\Users\W\OneDrive\Workspace\components\tspw

Screenshot 3: Create Application Component - Hello World
The 'Select Domain' tab is active. The 'standalone_microblaze_0' domain is selected. A 'Next' button is highlighted with a red box at the bottom right.

Screenshot 4: Create Application Component - Hello World
The 'Summary' tab is active. It displays the configuration for the new application component: Name: 'tspw', Workspace: 'C:\Users\W\OneDrive\Workspace', Component directory: 'C:\Users\W\OneDrive\Workspace\tspw', Platform: 'tspw', and Domain: 'standalone_microblaze_0 (OS:standalone, Processor:microblaze_0)'. A 'Finish' button is highlighted with a red box at the bottom right.

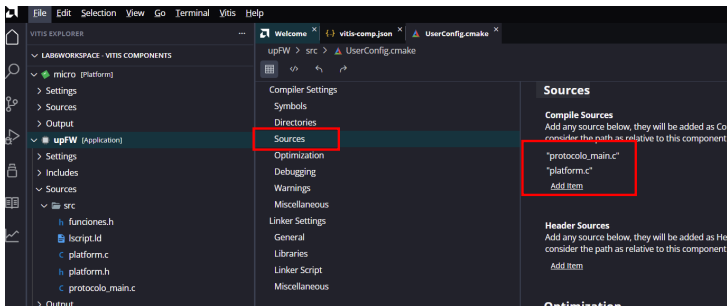
1- Definir el nombre del SW. 2- Seleccionar el componente que cargamos en los pasos anteriores. 3- Validar las configuraciones.

Crear la aplicación en Vitis



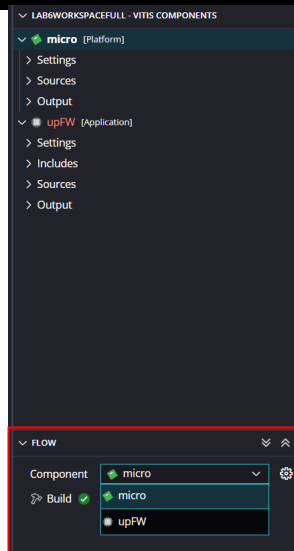
1- Reemplazar el archivo "Hello Word" con "protocolo_main" y "funciones.h" desde la dirección del proyecto. Ej. "/home/apola/workspace/MicroArti735/src/". 2- Seleccionar "UserConfig.cmake" para cambiar la referencia de compilación.

Crear la aplicación en Vitis



En la sección 'Source', reemplazar el nombre de Hello_Word por protocolo_main.

Crear la aplicación en Vitis



Seleccionar el componente y ejecutar la compilación. Repetir la compilación con la aplicación.

Túnel, Programar la FPGA y Ejecutar la Aplicación en Forma Remota



Tunel y Asignación de FPGA

- Conectarse al server: `ssh -L 3121:172.16.0.231:3121 user@181.231.212.3 -p 2222`
- Para el acceso se solicita usuario (**user**) y contraseña (**Cai1keaw**).
- Este acceso es para todos por igual.
- Ejecutar el script `client.py` para obtener el ID y el puerto USB de la FPGA.

```
Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ArielPola>ssh -L 3121:172.16.0.231:3121 user@181.231.212.3 -p 2222
user@181.231.212.3's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:   https://landscape.canonical.com
 * Support:      https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

49 updates can be applied immediately.
19 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

9 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Dec 11 10:35:45 2025 from 2.67.42.168
user@wanpu:~$
```

```
user@wanpu:~$ cd work_dda/apolaDDA2024/scripts/
user@wanpu:~/work_dda/apolaDDA2024/scripts$ python3 client.py
-----
Write-> Exit <-to close the session
-----
Ip: 127.0.0.1
-----
Port: 5005
-----
Checking Free Port
-----
USB Free: b'ttyUSB6-210319A28998A'
-----
Write Exit to close:
```

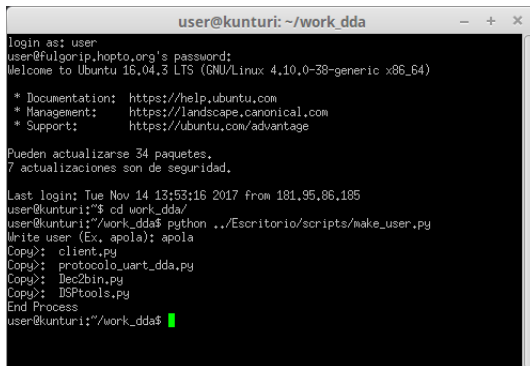
Tunel y Asignación de FPGA

- Para generar el entorno de trabajo debemos ejecutar la siguiente línea de comando dentro de la carpeta **work_dda** agregando el usuario personal con la inicial del nombre y el apellido completo (Ej. apola).

1 *cd work_dda*

2 *python3 ../Desktop/scripts/make_user.py*

- El script copia unos archivos y arma el árbol de carpetas.



```
user@kunturi: ~/work_dda
login as: user
user@fulgorip.hopto.org's password:
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-38-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

Pueden actualizarse 34 paquetes.
7 actualizaciones son de seguridad.

Last login: Tue Nov 14 13:53:16 2017 from 181.95.86.185
user@kunturi:~$ cd work_dda/
user@kunturi:~/work_dda$ python ../Escritorio/scripts/make_user.py
Write user (Ex. apola): apola
Copy>: client.py
Copy>: protocolo_uart_dda.py
Copy>: Idec2bin.py
Copy>: DSPtools.py
End Process
user@kunturi:~/work_dda$
```

Tunel y Asignación de FPGA

- Acceder a la carpeta `<user>/scripts/` y ejecutar `client.py` para solicitar un puerto USB y el ID de la placa FPGA disponible.

1 `cd apola/scripts`

2 `python3 client.py`

- El script retorna el USB: **USB3** y el ID de la FPGA: **210319A2CECCA**.
- En caso de no haber disponible ninguna placa, el script retorna que los puertos estan ocupados.
- Esta ventana y el script no se tienen que cerrar hasta no terminar de usar la FPGA, ya que se libera el puerto y dará acceso a otro usuario.
- En las siguientes figuras se observa el caso de asignación correcta y puertos ocupados.

Nota: Copia de archivos o carpetas

- Usamos el comando **pscp** para copiar archivos entre sesiones.

- Remoto a local

```
pscp -P 4466 user@fulgorip1.hopto.org:/home/user/work_dda/<user>/<file> ./
```

- Local a remoto

```
pscp -P 4466 ./<file> user@fulgorip1.hopto.org:/home/user/work_dda/<user>/
```

Script de configuración del Micro

```
configFpga.tcl X
scripts > configFpga.tcl > ...
1
2 # Complete with Project info
3 set FPGAID 210319A28998A
4 set BITNAME fpga
5 set APPNAME upFW
6 set HWNAME micro
7 set WORKSPACEPATH "C:/Users/ArielPola/lab6WorkspaceFull"
8 #####
9
10 puts "FPGA: $FPGAID"
11 puts "WorkSpace: $WORKSPACEPATH"
12
13 connect -url tcp:127.0.0.1:3121
14 targets -set -filter {jtag_cable_name =~ "Digilent Arty $FPGAID" && level==0 && jtag_device_ctx=="jsn-Arty-$FPGAID-0362d093-0"}
15 fpga -file $WORKSPACEPATH/$APPNAME/_ide/bitstream/$BITNAME.bit
16
17 targets -set -nocase -filter {name =~ "*microblaze*#0" && bscan=="USER2" && jtag_cable_name =~ "Digilent Arty $FPGAID" && jtag_device_ctx=="jsn-Arty-$FPGAID-0362d093-0"}
18 loadhw -hw $WORKSPACEPATH/$HWNAME/export/$HWNAME/hw/$BITNAME.xsa -regs
19 configparams mdm-detect-bscan-mask 2
20
21 targets -set -nocase -filter {name =~ "*microblaze*#0" && bscan=="USER2" && jtag_cable_name =~ "Digilent Arty $FPGAID" && jtag_device_ctx=="jsn-Arty-$FPGAID-0362d093-0"}
22 rst -system
23 after 3000
24
25 targets -set -nocase -filter {name =~ "*microblaze*#0" && bscan=="USER2" && jtag_cable_name =~ "Digilent Arty $FPGAID" && jtag_device_ctx=="jsn-Arty-$FPGAID-0362d093-0"}
26 dow $WORKSPACEPATH/$APPNAME/build/$APPNAME.elf
27
28 targets -set -nocase -filter {name =~ "*microblaze*#0" && bscan=="USER2" && jtag_cable_name =~ "Digilent Arty $FPGAID" && jtag_device_ctx=="jsn-Arty-$FPGAID-0362d093-0"}
29 con
```

Abrir el script en un editor y configurar los parametros según corresponda.

Script de configuración del Micro



En Vitis, abrir una consola y ejecutar el script.

Script de configuración del Micro

```
OUTPUT x C:\WINDOWS\SYSTEM32\CMD.EXE PROBLEMS 4 x

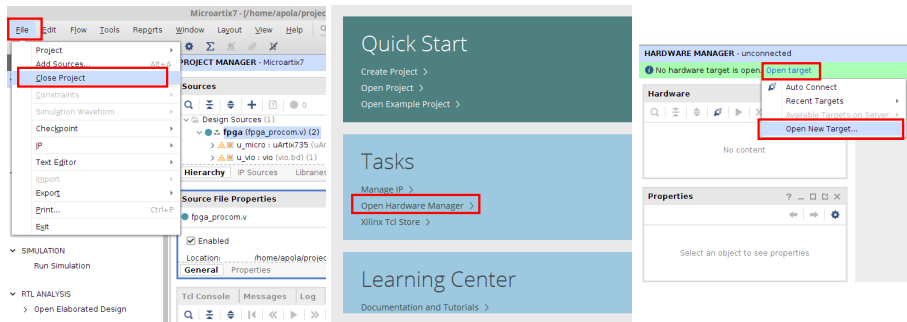
***** System Debugger (XSDB) v2024.2
**** Build date : Oct 29 2024-10:25:39
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2024 Advanced Micro Devices, Inc. All Rights Reserved.

    section, .vectors.sw_exception: 0x00000008 - 0x0000000f
    section, .vectors.interrupt: 0x00000010 - 0x00000017
    section, .vectors.hw_exception: 0x00000020 - 0x00000027
    section, .text: 0x00000050 - 0x0000021eb
    section, .init: 0x0000021ec - 0x000002227
    section, .fini: 0x000002228 - 0x000002247
    section, .ctors: 0x000002248 - 0x00000224f
    section, .dtors: 0x000002250 - 0x000002257
    section, .drvcfg_sec: 0x000002258 - 0x0000022bf
    section, .rodata: 0x0000022c0 - 0x0000022e9
    section, .sdata2: 0x0000022ea - 0x0000022ef
    section, .data: 0x0000022f0 - 0x00000234b
    section, .sdata: 0x00000234c - 0x00000234f
    section, .sbss: 0x000002350 - 0x00000234f
    section, .bss: 0x000002350 - 0x00000253f
    section, .heap: 0x000002540 - 0x000002d3f
    section, .stack: 0x000002d40 - 0x00000313f

100% 0MB 0.0MB/s 00:00
Setting PC to Program Start Address 0x00000000
Successfully downloaded C:/Users/ArielPola/lab6WorkspaceFull/upFW/build/upFW.elf
Info: MicroBlaze #0 (target 11) Running
xsdb% □
```

Se observa la palabra RUNNING al final del mensaje si el microprocesador listo para recibir comandos por USB.

Cargar parámetros del VIO



Volver a la ventana del Vivado, cerrar el proyecto, abrir el controlador de hardware y seleccionar un nuevo target.

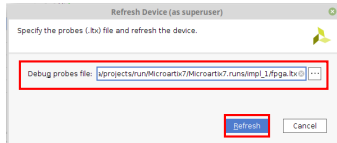
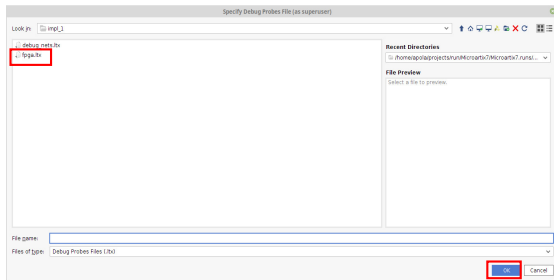
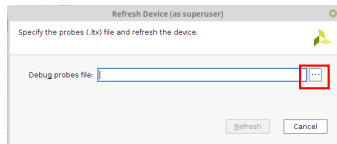
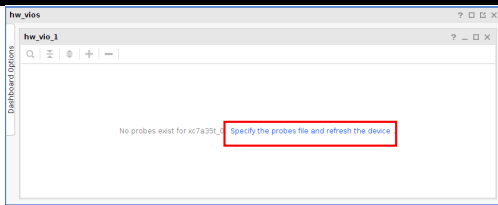
Cargar parámetros del VIO

The image displays four sequential screenshots of the Vivado IDE's 'Open New Hardware Target' wizard, illustrating the process of setting up a remote hardware server.

- Screenshot 1 (Top Left):** The 'Open Hardware Target' screen. It provides instructions on connecting to a hardware target. The 'Finish' button at the bottom is highlighted with a red box.
- Screenshot 2 (Top Right):** The 'Hardware Server Settings' screen. It prompts the user to select a local or remote hardware server. The 'Connect to' dropdown is set to 'Remote server (target is on remote machine)'. The 'Remote Server' section shows 'Host name' as 'localhost' and 'Port' as '3121'. This entire section is highlighted with a red box.
- Screenshot 3 (Bottom Left):** The 'Select Hardware Target' screen. It shows a list of available hardware targets. The target 'xilinx_tcf | Digilent/21031-9A28998A | 150000000' is selected and highlighted with a red box.
- Screenshot 4 (Bottom Right):** The 'Open Hardware Target Summary' screen. It displays the configured settings: 'Hardware Server Settings' (Server: localhost:3121) and 'Target Settings' (Target: xilinx_tcf/Digilent/21031-9A28998A, Frequency: 150000000). The 'Finish' button at the bottom is highlighted with a red box.

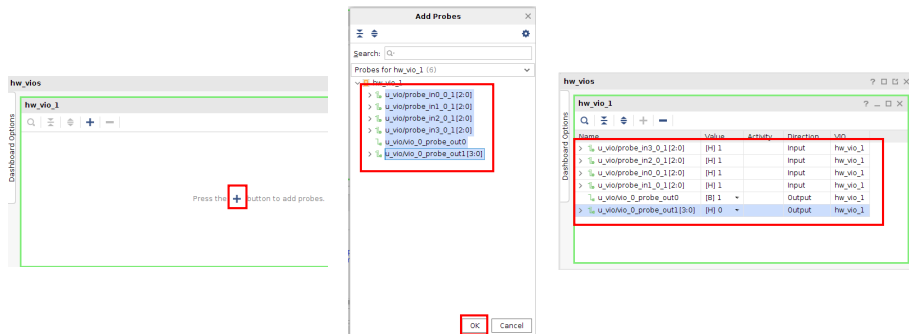
Crear un nuevo servidor cuya IP sea **LOCALHOST** y seleccionar la **FPGA** con el ID que previamente el servidor les entrego.

Cargar parámetros del VIO



*Debemos agregar el archivo con extensión **ltx** que se generó cuando creamos el VIO en los pasos anteriores. Se encuentra dentro de la carpeta < nameProject > .runs/impl_1/ < nameBinario > .ltx*

Cargar parámetros del VIO



Agregamos los puertos de entrada y salida que queremos controlar y observar.

Asignar el Puerto USB

A terminal window titled 'mc [user@wanpu]:/home/apola/Escritorio/Python' with standard window controls. The command '\$ python3 Python_UART_FPGA.py 5' is entered and highlighted with a red rectangular box. A green cursor is positioned at the end of the command.

```
mc [user@wanpu]:/home/apola/Escritorio/Python
$ python3 Python_UART_FPGA.py 5
```

*En un nuevo terminal de PUTTY, ejecutamos el script `Python_UART_FPGA.py` asignando el puerto UART. Ejecutar la aplicación con **python3***

VIO y Python

The image shows two side-by-side windows from a Vivado 2019.2 environment. The left window is the 'Hardware Manager' for a device 'localhost/xilinx_tcf/Digilent/210319A28998A'. It displays the 'hw_vios' properties for 'hw_vio_1'. A table lists various probes and their directions. The row for 'u_vio/vio_0_probe_out0' is highlighted, showing a value of '[B] 0' and a direction of 'Output'. The right window is a terminal showing a command prompt 'user@kunturi:~/work_dda/apola/scripts' with the command 'python Python_UART_FPGA.py' highlighted in a red box.

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 0		Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 0		Output	hw_vio_1

```
user@kunturi:~/work_dda/apola/scripts$ python Python_UART_FPGA.py
```

El puerto remarcado en el VIO es el que habilita si leemos el estado de los switch y lo que se carga desde el VIO (0: VIOCtrlEnb - 1: Switch). Ejecutamos el python3.

VIO y Python

The image shows two side-by-side windows. The left window is Vivado 2019.2 (as superuser) displaying the Hardware Manager. The right window is a terminal showing a Python script execution.

Vivado Hardware Manager:

Hardware Manager - localhost/xilinx_tcf/Digilent/Z10319A2B998A

hw_vios

hw_vio_1

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 1	↑	Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 1	↑	Input	hw_vio_1
> u_vio/probe_in_0_1[2:0]	[H] 1	↑	Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 1	↑	Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 0	↓	Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 0	↓	Output	hw_vio_1

Tcl Console:

```
commit_hw_vio [get_hw_probes (u_vio/vio_0_probe_out0)] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 1 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes (u_vio/vio_0_probe_out0)] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out1] -of_objects [get  
commit_hw_vio [get_hw_probes (u_vio/vio_0_probe_out1)] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes (u_vio/vio_0_probe_out0)] -of_objects [get_hw_vios -of
```

Type a Tcl command here

Terminal:

```
user@kunturi: ~/work_dda/apola/scripts  
user@kunturi:~/work_dda/apola/scripts$ python Python_URRT_FPGA.py  
None  
[Ingrese un comando:[0,1,2,3]]
```

Elegimos 0 y activa el color Rojo (1).

VIO y Python

The screenshot displays the Vivado 2019.2 (as superuser) interface. The 'Hardware Manager' tab is active, showing a table of hardware components for 'hw_vios'. The table has columns: Name, Value, Activity, Direction, and VIO. The 'Value' column for the first four rows is '[H] 2', which is highlighted with a red box. The 'Activity' column for these rows shows a green arrow icon. The 'Direction' column shows 'Input' and the 'VIO' column shows 'hw_vio_1'. The fifth row has 'Value' '[B] 0', 'Activity' a blue arrow icon, 'Direction' 'Output', and 'VIO' 'hw_vio_1'. The sixth row has 'Value' '[H] 0', 'Activity' a blue arrow icon, 'Direction' 'Output', and 'VIO' 'hw_vio_1'. Below the table is the 'Tcl Console' with a list of commands and a text input field.

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 2	↕	Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 2	↕	Input	hw_vio_1
> u_vio/probe_in0_0_1[2:0]	[H] 2	↕	Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 2	↕	Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 0	↗	Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 0	↗	Output	hw_vio_1

Tcl Console

```
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 1 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out1] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of_objects [get_hw_vios -of
```

Type a Tcl command here

Elegimos 1 y activa el color Verde (2).

VIO y Python

The screenshot displays the Vivado 2019.2 interface. The 'Hardware Manager' tab is active, showing a table of hardware components. The 'hw_vio_1' component is selected, and its properties are shown in a sub-window. The 'Value' column for the selected component is highlighted with a red box. The 'Tcl Console' is also visible, showing a list of commands and their outputs.

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 4		Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 4		Input	hw_vio_1
> u_vio/probe_in0_0_1[2:0]	[H] 4		Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 4		Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 0		Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 0		Output	hw_vio_1

The 'Tcl Console' shows the following commands and outputs:

```
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 1 [get_hw_probes u_vio/vio_0_probe_out0] -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out1] -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1}] -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out0] -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of objects [get_hw_vios -of
```

Elegimos 2 y activa el color Azul (4).

VIO y Python

The image shows two side-by-side windows. The left window is Vivado 2019.2 (as superuser) displaying the 'HARDWARE MANAGER' for 'localhost/xilinx_tcf/Digilent/210319A28998A'. It shows a table of VIO probes and outputs. The right window is a terminal showing a Python script being executed.

Vivado Hardware Manager Table:

Name	Value	Activity	Direction	VIO
u_vio/probe_in3_0_1[2:0]	[H] 0	↓	Input	hw_vio_1
u_vio/probe_in2_0_1[2:0]	[H] 0	↓	Input	hw_vio_1
u_vio/probe_in0_0_1[2:0]	[H] 0	↓	Input	hw_vio_1
u_vio/probe_in1_0_1[2:0]	[H] 0	↓	Input	hw_vio_1
u_vio/vio_0_probe_out0	[B] 0	↓	Output	hw_vio_1
u_vio/vio_0_probe_out1[3:0]	[H] 0	↓	Output	hw_vio_1

Terminal Output:

```
user@kunturk: ~/work_dda/apola/scripts
user@kunturk:~/work_dda/apola/scripts$ python Python_UART_FPGA.py
None
Ingresar un comando[0,1,2,3]
<< 0
<< 1
<< 2
<< 3
Wait Input Data
0
>> 0
<< 0
```

Elegimos 3 para leer el estado de los Switch. Con la salida out0: 0 habilitamos el control desde el VIO y el python tiene que leer el estado de la salida out1 (Python >> 0).

VIO y Python

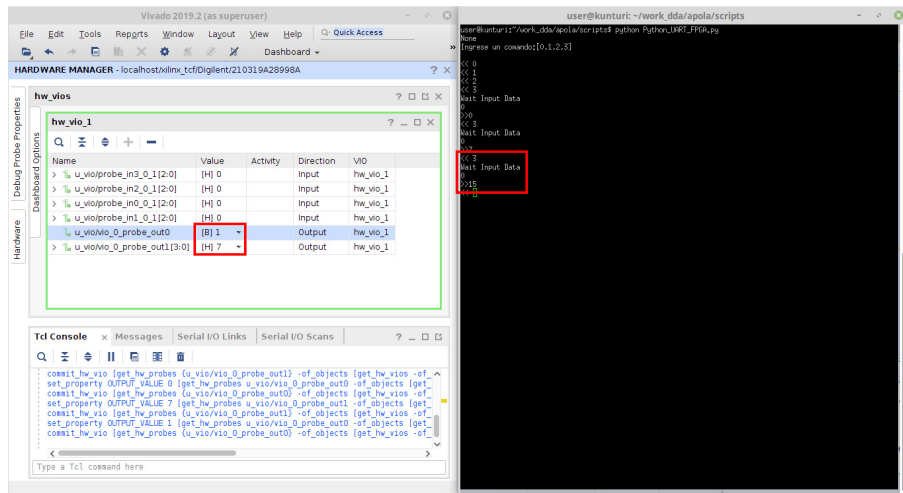
The image shows two windows side-by-side. The left window is Vivado 2019.2 (as superuser) displaying the 'HARDWARE MANAGER' for 'localhost/xilinx_tcf/Digilent/210319A28998A'. It shows a table of hardware probes for 'hw_vio_1'. The table has columns: Name, Value, Activity, Direction, and VIO. The rows are:

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 0		Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 7		Output	hw_vio_1

The 'Value' column for the last two rows is highlighted with a red box. Below the table is the 'Tcl Console' showing a series of commands to configure the hardware probes. The right window is a terminal window titled 'user@kunturi: ~/work_dda/apola/scripts'. It shows the command 'python Python_UART_FPGA.py' being executed, followed by the output 'None' and 'Ingresar un comando:[0,1,2,3]'. The terminal also shows a series of '<<' and '>>' characters, and a red box highlights the 'Wait Input Data' prompt.

Elegimos 3 para leer el estado de los Switch. Con la salida out0: 0 habilitamos el control desde el VIO y el python tiene que leer el estado de la salida out1 (Python >> 7).

VIO y Python



The screenshot shows the Vivado 2019.2 interface. The Hardware Manager window displays a table of VIO probes and outputs. The table has columns: Name, Value, Activity, Direction, and VIO. The rows are:

Name	Value	Activity	Direction	VIO
u_vio/probe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
u_vio/probe_in2_0_1[2:0]	[H] 0		Input	hw_vio_1
u_vio/probe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
u_vio/probe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
u_vio/vio_0_probe_out0	[B] 1		Output	hw_vio_1
u_vio/vio_0_probe_out1[3:0]	[H] 7		Output	hw_vio_1

The Tcl Console shows the following commands:

```
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1} -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out0 -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0} -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 7 [get_hw_probes u_vio/vio_0_probe_out1 -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1} -of objects [get_hw_vios -of  
set_property OUTPUT_VALUE 1 [get_hw_probes u_vio/vio_0_probe_out0 -of objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0} -of objects [get_hw_vios -of
```

The Python script in the background shows the following code:

```
user@kunturi: ~/work_dda/apola/scripts$ python Python_UART_FPGA.py  
None  
Ingrese un comando[0,1,2,3]  
<< 0  
<< 1  
<< 2  
<< 3  
Wait Input Data  
>>0  
<< 3  
Wait Input Data  
>>15  
>>15
```

Elegimos 3 para leer el estado de los Switch. Con la salida out0: 1 habilitamos el control desde el VIO y el python tiene que leer el estado de los switches (Python >> 15).

VIO y Python

The image shows a Vivado 2019.2 interface on the left and a terminal window on the right. The Vivado window displays the 'HARDWARE MANAGER' for a local host, showing a table of hardware probes and outputs. The terminal window shows a Python script being executed, with the prompt 'user@kunturi:~/work_dda/apola/scripts\$' highlighted by a red box.

Vivado 2019.2 (as superuser)

HARDWARE MANAGER - localhost/xilinx_tcf/Digilent/210319A28998A

hw_vios

Name	Value	Activity	Direction	VIO
> u_vio/probe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in2_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/probe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
> u_vio/vio_0_probe_out0	[B] 1		Output	hw_vio_1
> u_vio/vio_0_probe_out1[3:0]	[H] 7		Output	hw_vio_1

Tcl Console

```
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 0 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 7 [get_hw_probes u_vio/vio_0_probe_out1] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out1}] -of_objects [get_hw_vios -of  
set_property OUTPUT_VALUE 1 [get_hw_probes u_vio/vio_0_probe_out0] -of_objects [get  
commit_hw_vio [get_hw_probes {u_vio/vio_0_probe_out0}] -of_objects [get_hw_vios -of
```

Terminal Window

```
user@kunturi:~/work_dda/apola/scripts$ python Python_UART_FPGA.py  
None  
Ingresar un comando:[0,1,2,3]  
<< 0  
<< 1  
<< 2  
<< 3  
Wait Input Data  
0  
>> 0  
<< 3  
Wait Input Data  
0  
>> 7  
<< 3  
Wait Input Data  
0  
>> 6  
<< exit  
user@kunturi:~/work_dda/apola/scripts$
```

Salimos del script.

Cierre de Sesiones

Cerrando el Tunel

- Liberando el puerto USB
 - Para liberar el puerto USB asignado escribir la palabra **Exit**.
- Cerrando los dos tuneles
 - Para cerrar ambas sesiones escribir la palabra **exit**.

The image shows two terminal windows side-by-side. The left window shows the output of a script that sets up a USB tunnel. It displays the IP address 127.0.0.1 and port 5005. It then prompts the user to write text to check the session or exit to close. The user enters 'Exit', which is echoed back. The script then prompts to close the session, and the user enters 'exit'. The right window shows the output of a script that sets up a UART tunnel. It displays the IP address 127.0.0.1 and port 5005. It then prompts the user to write text to check the session or exit to close. The user enters 'Exit', which is echoed back. The script then prompts to close the session, and the user enters 'exit'.

```
user@kunturi: ~/work_dda/apola/scripts
* Support: https://ubuntu.com/advantage
Pueden actualizarse 27 paquetes.
0 actualizaciones son de seguridad.
Last login: Wed Nov 15 21:11:43 2017 from 200.126.231.123
user@kunturi:~$ cd work_dda/apola/scripts/
user@kunturi:~/work_dda/apola/scripts$ python client.py
Write-> Exit <-to close the session
Ip: 127.0.0.1
Port: 5005
Checking Free Port
USB Free: USB3-210319A2CECCA
Write Text to check Session or Exit to close: Exit
Echo: Exit
Close Session
user@kunturi:~/work_dda/apola/scripts$ exit

user@kunturi: ~/work_dda/apola/scripts
Welcome to Ubuntu 16.04.3 LTS (GNU/Linux 4.10.0-38-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
Pueden actualizarse 27 paquetes.
0 actualizaciones son de seguridad.
Last login: Wed Nov 15 22:33:00 2017 from 200.126.231.123
user@kunturi:~$ cd work_dda/apola/scripts/
user@kunturi:~/work_dda/apola/scripts$ python protocolo_uart_dda.py 3
RESET ON
MODULES OFF
LOG RUN OFF
RESET OFF
ENB MODULES
LOG
LOG RUN OFF
LOG RUN ON
LOG READ
4096
End Script
user@kunturi:~/work_dda/apola/scripts$ exit
```

Las figuras muestran como liberar el USB asignado y el cierre de los tuneles abiertos.